

Reinforcement Learning (Обучение с подкреплением)

Daniil Zelezetsky

Higher School of Economics

April 3, 2025

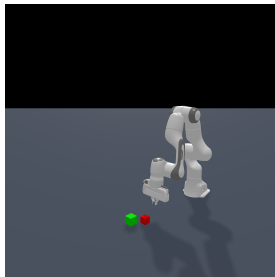
About Me



- Даниил Зелезецкий
- Аспирант и исследователь
Центра Когнитивного
Моделирования МФТИ
- Научные интересы: Обучение с
подкреплением на базе
трансформерных архитектур в
задачах робототехники
- ТГ @zelezetsky

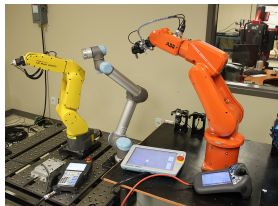


Мотивация



- 1 RNN, CNN и т.д. это мощный инструмент для решения широкого класса задач, однако кажется, что всё сводится к задаче регрессии и классификации в **статике** (отличи кошку от собаки, предскажи следующее слово в предложении).
- 2 Не совсем понятно, как научить нейросеть принимать решения в динамике.
- 3 Как заставить нейросеть играть в Ping-Pong или Doom, решая всю ту же задачу классификации?

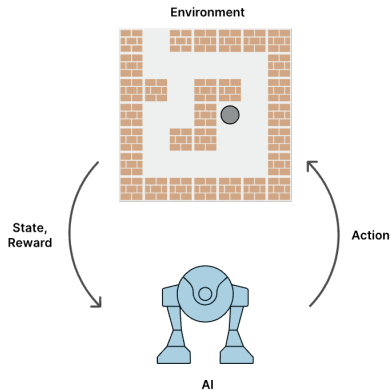
Решение - Reinforcement Learning (RL)



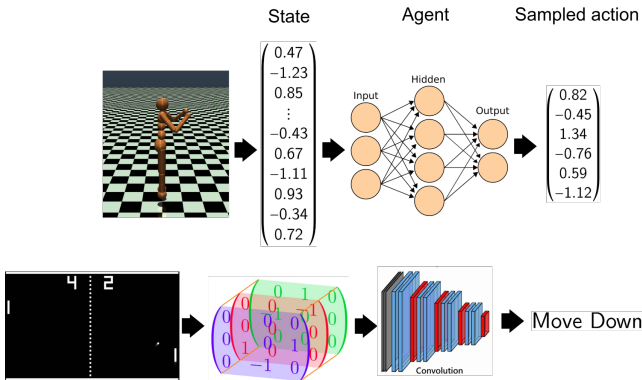
- **Робототехника:** Манипуляционные роботы, адаптирующиеся к изменяющимся условиям и непредсказуемым ситуациям.
- **Игры и симуляции:** Разработка алгоритмов, способных обучаться стратегии в играх.
- **Автономные автомобили:** Автопилот Tesla, обучающийся безопасному вождению.
- **Финансовые рынки:** Алгоритмическая торговля и управление портфелем.

Решение - Reinforcement Learning (RL)

- 1 Агент π находясь в среде наблюдает своё состояние s_t .
- 2 Приняв решение, агент совершает действие a_t в среде.
- 3 В ответ на это действие, среда возвращает агенту вознаграждение r_t и новое состояние s_{t+1} , в которое агент перешел.



Как подавать состояние s в нейронную сеть



Марковский процесс принятия решений (MDP)

Математическая модель, описывающая процесс взаимодействия со средой.

Компоненты MDP:

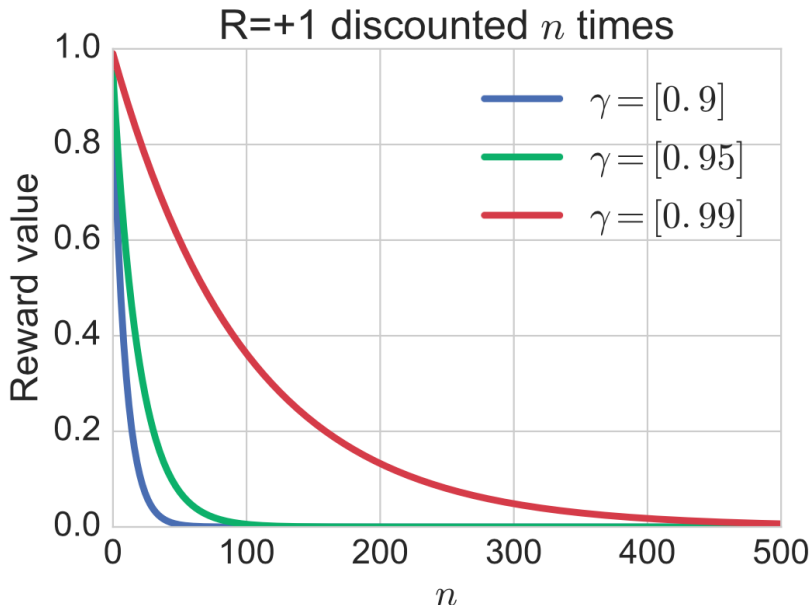
- $\mathbf{S}$ — множество состояний (states)
- $\mathbf{A}$ — множество действий (actions)
- $\mathbf{P}(s' \mid s, a)$ — функция переходов (вероятность попасть в состояние s' при выполнении действия a в состоянии s).
- $\mathbf{R}(s, a, s')$ — функция награды (reward)
- γ — коэффициент дисконтирования (discount factor), $\gamma \in [0, 1]$

Задача агента: Найти стратегию $\pi(s)$, которая максимизирует ожидаемую совокупную награду:

$$\pi^*(s) = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \right]$$

Важно заметить, что $P(s_{t+1} | s_t, a_t) = P(s_{t+1} | s_0, a_0, s_1, a_1, \dots, s_t, a_t)$

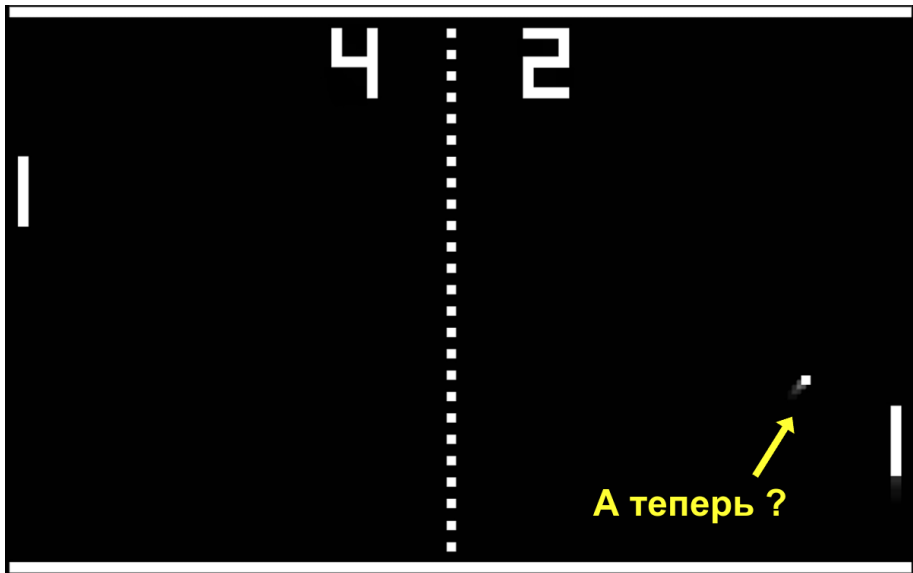
Gamma



Ограничения марковского процесса



Ограничения марковского процесса



Partially Observable Markov Decision Process (POMDP)

Определение POMDP: POMDP (Частично наблюдаемый марковский процесс принятия решений) — расширение MDP, где агент не может точно наблюдать состояние среды, а имеет лишь вероятностные наблюдения.

Компоненты POMDP:

- S — множество состояний (states)
- A — множество действий (actions)
- O — множество наблюдений (observations)
- $P(s' | s, a)$ — функция переходов
- $R(s, a, s')$ — функция награды
- $Z(o | s')$ — функция наблюдений (вероятность получить наблюдение o в состоянии s')
- γ — коэффициент дисконтирования (discount factor)

Свойство POMDP:

$$P(s_{t+1}, o_{t+1} | s_t, a_t) = P(s_{t+1} | s_t, a_t) \cdot P(o_{t+1} | s_{t+1})$$

Подходы к обучению агента

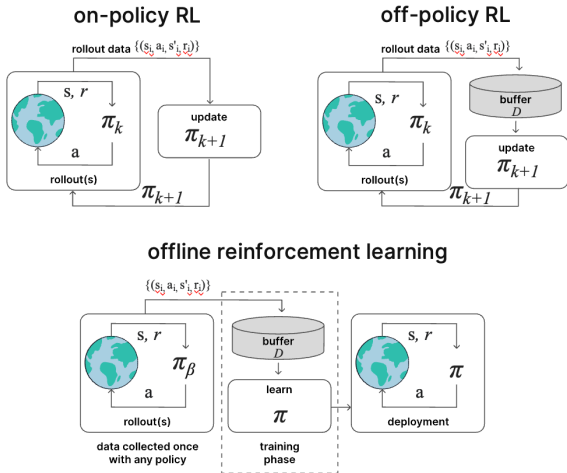
1 Off-Policy RL:

Собираем отложенный опыт, на котором учимся (DQN, DDPG, SAC)

2 On-Policy RL:

Обучаем стратегию по мере исследования среды (REINFORCE, AC, PPO)

3 Offline RL: Учимся на оффлайн датасете экспертных данных (обычный sequence modeling)



Функция состояния V

В контексте обучения с подкреплением функция состояния $V(s)$ определяет ожидаемую сумму наград, которую агент получит, начиная с состояния s и следуя определенной стратегии π .

$$V^{\pi}(s) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right]$$

где:

- s — текущее состояние,
- r_t — награда на шаге t ,
- γ — коэффициент дисконтирования (от 0 до 1),
- π — стратегия агента.

Функция состояния-действия Q

Функция состояния-действия $Q(s, a)$ оценивает ожидаемую сумму наград, которую агент получит, начиная в состоянии s , выбирая действие a , и следуя стратегии π .

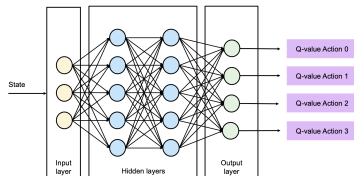
$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right]$$

Тогда можно вывести Q через V :

$$Q^\pi(s_0, a_0) = \gamma^0 r(s_0, a_0) + V^\pi(s_1)$$

где:

- s — текущее состояние,
- a — выбранное действие,
- r_t — награда на шаге t ,
- γ — коэффициент дисконтирования,
- π — стратегия агента.



Что такое DQN? Deep Q-Network (DQN) — это алгоритм обучения с подкреплением, который объединяет Q-Learning и нейронные сети. Используется для обучения агентов в средах с большими пространствами состояний.

Основные идеи:

- Использование опыта (experience replay) для повышения стабильности.
- Применение целевой сети (target network) для уменьшения колебаний при обучении.

Q-функция в DQN

Обновление Q-функции в DQN:

- Q-функция приближается нейросетью с параметрами θ .
- Целевая функция:

$$L(\theta) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1})} \left[(y_t - Q^\theta(s_t, a_t))^2 \right]$$

где

$$y_t = r_t + \gamma \max_{a'} Q^{\theta^-}(s_{t+1}, a')$$

- θ^- — параметры целевой сети (target network), которые обновляются реже.

Temporal Difference Error:

$$TD(\theta) = \frac{1}{N} \sum_{(s_t, a_t, r_t, s_{t+1}) \in D} \left[\left(r_t + \gamma \max_{a'} Q^{\theta^-}(s_{t+1}, a') - Q^\theta(s_t, a_t) \right)^2 \right]$$

Проблема коррелированных данных:

- Обучение на последовательных данных вызывает корреляции и нестабильность.

Решение: Experience Replay Buffer

- Хранение опыта (s_t, a_t, r_t, s_{t+1}) в буфере памяти.
- На каждом шаге обучения берётся случайная выборка из буфера.
- Разрывает корреляции и стабилизирует обучение.

Проблема расхождения при обучении:

- Обновление нейросети с самоподдерживающимися таргетами ведёт к расхождению.

Решение: Target Network

- Целевая сеть $Q^{\theta^-}(s, a)$ копирует параметры основной сети $Q^{\theta}(s, a)$ через фиксированные интервалы (как правило раз в два шага).
- Снижает вероятность расхождения алгоритма.

ε-жадная стратегия в DQN

Проблема исследования среды:

- Если каждый раз в среде выбирать то действие a , которое максимизирует $Q(s, a)$, то появляются риски плохого исследования среды.
- Плохое исследование среды влечёт за собой неоптимальное действие в состояниях, которые раньше не встречались стратегии.
- Иными словами, нам важно побывать в различных состояниях, пускай даже и плохих для агента.

Решение: ε-жадная стратегия

- Вместо жадного выбора наилучшего действия, используем ε-жадную стратегию.
- На каждом шаге взаимодействия со средой, с вероятностью $1 - \epsilon$ выбираем действие жадно, а с вероятностью ϵ случайно.
- Это позволит нам качественнее исследовать среду и достигать стабильного перформанса агента.

Алгоритм DQN

Алгоритм DQN:

- ❶ Инициализация сети $Q^\theta(s, a)$ случайными весами.
- ❷ Инициализация target-сети $Q^{\theta^-}(s, a) = Q^\theta(s, a)$.
- ❸ Для каждого эпизода среде:
 - Выполняем действие a_t с ϵ -жадной стратегией в ответ на наблюдаемое состояние s_t и получаем в ответ от среды r_t и s_{t+1} .
 - Сохраняем переход (s_t, a_t, r_t, s_{t+1}) в буфер опыта D .
 - Сэмплируем случайную выборку из буфера и считаем loss функцию:

$$L(\theta) = \frac{1}{N} \sum_{(s_t, a_t, r_t, s_{t+1}) \in D} \left[\left(r_t + \gamma \max_{a'} Q^{\theta^-}(s_{t+1}, a') - Q^\theta(s_t, a_t) \right)^2 \right]$$

- Обновляем веса сети по анти-градиенту функции потерь:

$$\theta^{new} = \theta^{old} - \alpha \nabla_{\theta} L(\theta^{old})$$

- Каждые N шагов обновляем target-сеть:

$$\theta^- \leftarrow \theta$$

Преимущества и Недостатки DQN

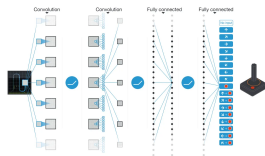
Преимущества:

- Обработка сложных сред с большим пространством состояний.
- Использование нейросетей для обучения в реальном времени.

Недостатки:

- Может быть нестабильным или расходиться без модификаций (Double DQN, Dueling DQN).
- Плохо масштабируется на задачи с непрерывным пространством действий.

Заключение



Что мы узнали о DQN:

- Представив DQN в 2013 году, DeepMind доказали, что нейросети могут эффективно использоваться в RL.
- DQN — мощный алгоритм для задач с дискретными действиями.
- Использует experience replay и target network для стабильности.
- Ложит основу для более продвинутых алгоритмов (Double DQN, Dueling DQN, Rainbow DQN).